CODE REVIEW

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 4: Treat Findings as Hypotheses

- When the AI produces review comments, treat each one as a claim you still have to verify.
- Expect a meaningful fraction to be wrong or unimportant.
- For an independent second check, run the AI blind without showing it prior human comments and then use a primed pass only for consolidation.

Reasoning: Why Findings Need Verification

- LLM hallucinations and valid-but-unimportant comments are two distinct false-positive sources
- LLM review precision falls below 10% in many full-PR configurations
- Reviewers who saw a prior comment were 62% likely to find the same bug type again vs. 11% for unprimed reviewers
- Pasting human comments into the AI prompt silently removes the independence of the second check

Key Practices: Making Findings Verifiable

- Require each finding to cite the exact line or lines it is about
- Require a short evidence field for each finding: which tokens or control-flow paths support the claim
- Ask the AI to produce a minimal failing example (input or trace) where possible
- A finding you cannot reproduce is a finding you cannot trust

Key Practices: Triaging and Pass Independence

- For each comment, locate the evidence in the code and if you cannot, treat the comment as suspect
- Classify each comment: confirmed defect, style preference, false positive, or unknown
- Do not batch-accept "apply all suggestions" as that hides false positives inside correct ones
- For independent second-opinion checks, run the AI blind: do not paste the human review thread into the prompt
- Run the primed pass second, for consolidation only: "here are human comments so far; have they missed any related issues?"

Guideline 5: Own the Merge, Own the Learning

- Keep the merge decision, the approve click, and the responsibility for what ships with a human.
- Do not let AI consume the non-defect benefits of review: code understanding, learning from the author, and shared ownership of the change.
- If you skim only the AI summary and skip the diff, review has become a rubber stamp.

Reasoning: Accountability and Knowledge Transfer

- LLM assistance disrupts accountability because "one cannot be accountable to an LLM"
- Four drivers of quality: personal standards, professional integrity, pride, and reputation only operate between people
- The developer who hits Merge is accountable; AI cannot hold that role
- Automation bias causes people to miss errors at higher rates when following automated recommendations
- Reviews provide more value through knowledge transfer and shared ownership than defect detection alone
- Fully automating review risks losing the majority of interpersonal benefits such as shared ownership

Key Practices: Prompting AI to Support Your Judgment

- Ask for observations with evidence, not verdicts and avoid "is this safe to merge?"
- Ask the AI to rank its comments by confidence and flag any point where it had to guess
- Ask for alternative approaches the author could have taken, not just defects
- Ask the AI to explain why each issue matters and use these as raw material for your own review comments

Key Practices: Preserving What Review Is Actually For

- Read the diff yourself at least once before looking at AI comments and if you read only the AI summary, you have not learned the module
- When writing review comments, explain the why, not just the what as useful comments teach
- Own the sign-off: the approve click is a personal claim that you read, understood, and accept responsibility for the change
- Respond to the author yourself as the back-and-forth is the knowledge transfer
- For critical modules, a human reviewer should still be in the loop even when AI review is available

Guideline 6: Apply Heightened Scrutiny for AI-Authored and Security-Sensitive Code

- Default-level AI review is calibrated for routine code while some changes need more.
- AI-authored code looks polished but contains more latent defects than human-authored code on average.
- For security-sensitive changes, layer the AI reviewer on top of static analysis and give it a named threat list rather than a vague "is this safe?" prompt.

Reasoning: Why AI Code Needs Extra Scrutiny

- AI-authored PRs have ~1.7x more issues on average; logic defects up 75%, security vulnerabilities up 1.5 to 2x
- AI code is simpler but more prone to unused constructs, hardcoded debugging artifacts, and high-risk vulnerabilities
- AI assistance cut trivial syntax errors 76% but caused a 10x surge in architectural and security findings
- LLM-based malicious-PR detectors achieve 99.3% accuracy when run alongside static analysis against a named threat list
- Security-relevant comments improve substantially when the prompt includes call-site and enclosing-function context

Key Practices: Recognizing When Heightened Scrutiny Is Needed

- PR description mentions co-authored-with-AI, @claude, @copilot, or similar
- PR touches authentication, authorization, cryptography, file/subprocess I/O, CI config, build scripts, or dependency manifests
- PR comes from a first-time contributor to an open-source project
- PR is unusually large or touches many subsystems at once (also a Guideline 1 failure, but now with a security dimension)

Key Practices: What Heightened Scrutiny Looks Like

- Give the AI a named threat list (e.g., for CI: secret exfiltration, unpinned actions, pull_request_target misuse)
- If the code is AI-generated, ask it to list every external symbol and confirm each one actually exists
- Run static analysis and dependency scanners first as AI is most useful on patterns these miss
- Do not ask "is this safe?" instead ask for evidence per threat with severity (low/medium/high)
- A human reads any change touching auth, cryptography, subprocess, or CI regardless of tool findings

References [1-5]

- [1] Alami et al. (2025). Accountability in Code Review: The Role of Intrinsic Drivers and the Impact of LLMs. ACM TOSEM. DOI: 10.1145/3721127.
- [2] Kawalerowicz et al. (2026). LLMs as Code Review Agents: A Rapid Review and Experimental Evaluation. Springer LNNS. DOI: 10.1007/978-3-032-09318-9_24.
- [3] Adalsteinsson et al. (2025). Rethinking Code Review Workflows with LLM Assistance: An Empirical Study. arXiv:2505.16339.
- [4] Ramesh et al. (2025). Automated Code Review Using LLMs at Ericsson: An Experience Report. ICSME 2025. arXiv:2507.19115.
- [5] Haider et al. (2024). Prompting and Fine-tuning LLMs for Automated Code Review Comment Generation. arXiv:2411.10129.

References [6-10]

- [6] Zhang et al. (2025). LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM. ASE 2025. arXiv:2512.01356.
- [7] GitHub Engineering. (2025). Code review in the age of AI: Why developers will always own the merge button. The GitHub Blog.
- [8] Uber Engineering. (2025). uReview: Scalable, Trustworthy GenAI for Code Review at Uber. Uber Blog.
- [9] Romeo and Conti. (2025). Exploring automation bias in human-AI collaboration. AI & Society. DOI: 10.1007/s00146-025-02422-7.
- [10] GitHub Docs. (2026). Using custom instructions to unlock the power of Copilot code review. docs.github.com.

References [11-15]

- [11] GitHub Engineering. (2025). Unlocking the full power of Copilot code review. The GitHub Blog.
- [12] Cihan et al. (2025). Evaluating Large Language Models for Code Review. arXiv:2505.20206.
- [13] Zeng et al. (2025). Benchmarking and Studying the LLM-based Code Review. arXiv:2509.01494.
- [14] CodeRabbit. (2025). State of AI vs Human Code Generation Report. coderabbit.ai.
- [15] Cotroneo et al. (2025). Human-Written vs. AI-Generated Code: A Large-Scale Study. arXiv:2508.21634.

References [16-20]

- [16] Apiiro. (2025). 4x Velocity, 10x Vulnerabilities: AI Coding Assistants Are Shipping More Risks. Apiiro Blog.
- [17] Liu et al. (2024). Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. arXiv:2404.00971.
- [18] Lamb et al. (2025). Detecting malicious pull requests at scale with LLMs. Datadog Engineering Blog.
- [19] Bacchelli and Bird. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. ICSE. DOI: 10.1109/ICSE.2013.6606617.
- [20] Rigby and Bird. (2013). Convergent Contemporary Software Peer Review Practices. ESEC/FSE. DOI: 10.1145/2491411.2491444.

References [21-25]

- [21] Kemerer and Paulk. (2009). The Impact of Design and Code Reviews on Software Quality. IEEE TSE. DOI: 10.1109/TSE.2009.27.
- [22] Sadowski et al. (2018). Modern Code Review: A Case Study at Google. ICSE-SEIP. DOI: 10.1145/3183519.3183525.
- [23] Bosu et al. (2015). Characteristics of Useful Code Reviews: An Empirical Study at Microsoft. MSR. DOI: 10.1109/MSR.2015.21.
- [24] Spadini et al. (2020). Primers or Reminders? The Effects of Existing Review Comments on Code Review. ICSE. DOI: 10.1145/3377811.3380385.
- [25] Heander et al. (2025). Code Review as Decision-Making: Building a Cognitive Model from the Questions Asked During Code Review. arXiv:2507.09637.